

BACCALAURÉAT GÉNÉRAL

ÉPREUVE D'ENSEIGNEMENT DE SPÉCIALITÉ

Session 2025

NUMÉRIQUE ET SCIENCES INFORMATIQUES

Amérique du Nord 2025 – Jour 1

DURÉE DE L'ÉPREUVE: 3 heures 30

*Ce sujet comporte 8 pages numérotées de 1/8 à 8/8
La dernière page comporte une annexe à remettre complétée de votre nom et classe.
Le candidat doit traiter les trois exercices proposés.*

L'utilisation d'une calculatrice n'est pas autorisée

Exercice 1 ▶

/6 points

1. OUI – SORBIER
2. NON (-)
3. Implémentation de `arbre_2`

</> Code Python

```
1 F1 = Feuille_resultat([])
2 F2 = Feuille_resultat(['Sorbier'])
3 F3 = Feuille_resultat(['Robinier', 'Noyer'])
4 F4 = Feuille_resultat([])
5
6 N1 = Noeud('Bord denté', F2, F3 )
7 N2 = Noeud('Alternées ?', N1, F4)
8 N3 = Noeud('Simples ?', F1, N2)
9
10 arbre_2 = N3
```

4. Méthode `est_resultat` de la classe `Noeud`:

</> Code Python

```
1 def est_resultat(self) -> bool:
2     """Retourne False car un noeud n'est pas un résultat"""
3     return False
```

5. Méthode `est_resultat` de la classe `Feuille_resultat`:

</> Code Python

```
1 def est_resultat(self) -> bool:
2     """Retourne True car une feuille est un résultat"""
3     return True
```

6. Méthode `nb_vegetaux` pour la classe `Feuille_resultat`:

</> Code Python

```
1 def nb_vegetaux(self) -> int:
2     """Retourne le nombre de végétaux dans cette feuille"""
3     return len(self.vegetaux)
```

7. Méthode `nb_vegetaux` pour la classe `Noeud`:

</> Code Python

```
1 def nb_vegetaux(self) -> int:
2     """Retourne le nombre total de végétaux identifiables
3     à partir de ce noeud"""
4     return self.sioui.nb_vegetaux() + self.sinon.nb_vegetaux()
```

8. Méthode `liste_questions` pour la classe `Feuille_resultat`:

</> Code Python

```
1 def liste_questions(self) -> list:
2     """Retourne une liste vide car une feuille ne contient pas de questions"""
3     return []
```

9. Méthode `liste_questions` pour la classe `Noeud`:

</> Code Python

```
1 def liste_questions(self) -> list:
2     """Retourne la liste de toutes les questions accessibles
3     à partir de ce noeud"""
4     return [self.question] +
5             self.sioui.liste_questions() +
6             self.sinon.liste_questions()
```

10. Fonction `est_bien_renseigne`

</> Code Python

```
1 def est_bien_renseigne(dico_vegetal: dict, arbre) -> bool:
2     """
3     True si toutes les questions de l'arbre sont des clés du dictionnaire,
4     False sinon
5     """
6     questions_arbre = arbre.liste_questions() # list
7     for q in questions_arbre:
8         if q not in dico_vegetal.keys():
9             return False
10    return True
```

11. Fonction `identifier_vegetaux`

</> Code Python

```
1 def identifier_vegetaux(dico_vegetal: dict, arbre) -> list:
2     """
3     Returns:
4     Liste des noms des végétaux identifiés (peut être vide)
5     """
6     # Si on est arrivé à une feuille, retourner les végétaux
7     if arbre.est_resultat():
8         return arbre.vegetaux
9     # Si on est sur un noeud, suivre la branche correspondant à la réponse
10    question = arbre.question
11    reponse = dico_vegetal[question]
12    if reponse:
13        return identifier_vegetaux(dico_vegetal, arbre.sioui)
14    else:
15        return identifier_vegetaux(dico_vegetal, arbre.sinon)
```

Exercice 2 ▶

/6 points

1. Méthode passer_transit

</> Code Python

```
1 # Question 1 : Méthode pour changer l'état à 'transit'
2 def passer_transit(self):
3     """Change l'état du colis à 'transit'"""
4     self.etat = 'transit'
```

2. Fonction ajouter_colis

</> Code Python

```
1 # Question 2 : Fonction ajouter_colis modifiée avec vérification du poids
2 def ajouter_colis(liste: list, colis: Colis):
3     """Ajoute un colis à la liste si son poids est <= 25kg,
4     sinon affiche un message d'erreur"""
5     if colis.poids <= 25:
6         liste.append(colis)
7     else:
8         print("Dépassement du poids maximal autorisé")
```

3. Fonction nb_colis

</> Code Python

```
1 # Question 3 : Fonction pour compter le nombre de colis dans une liste
2 def nb_colis(liste: list) -> int:
3     """Renvoie le nombre de colis dans la liste"""
4     return len(liste)
```

4. Fonction poids_total

</> Code Python

```
1 # Question 4 : Fonction pour calculer le poids total
2 def poids_total(liste: list) -> int:
3     """Renvoie le poids total de tous les colis dans la liste"""
4     total = 0
5     for c in liste:
6         total = total + c.poids
7     return total
```

5. Fonction liste_colis_etat

</> Code Python

```
1 # Question 5 : Fonction pour filtrer les colis par état
2 def liste_colis_etat(liste: list, statut: str) -> list:
3     """Renvoie une nouvelle liste contenant les colis ayant l'état spécifié"""
4     nouvelle_liste = []
5     for colis in liste:
6         if colis.etat == statut:
7             nouvelle_liste.append(colis)
8     return nouvelle_liste
```

6. La fonction tri_decroissant utilise le TRI PAR SÉLECTION. Complexité dans le pire des cas : $O(n^2)$.
7. On pourrait utiliser le TRI FUSION (MERGE SORT). Complexité dans le pire des cas : $O(n \log n)$. meilleure que le tri par sélection

8. Fonction chargement_glouton

```
</> Code Python
1 def chargement_glouton(liste: list, rang: int, capacite: int) -> list:
2     """
3     Renvoie :
4         liste de colis à charger en utilisant un algorithme glouton
5     """
6     # Cas de base : si tous les colis ont été examinés
7     if rang == len(liste):
8         return []
9     # Si le colis courant rentre dans la capacité restante
10    elif liste[rang].poids <= capacite:
11        return [liste[rang]] +
12            chargement_glouton(liste, rang + 1, capacite - liste[rang].poids)
13    # Sinon, on le saute
14    else:
15        return chargement_glouton(liste, rang + 1, capacite)
```

9. Une `RecursionError` se produit lorsque la profondeur maximale de récursion est atteinte. Cela arrive lorsque la fonction récursive s'appelle trop de fois sans atteindre un cas de base, typiquement avec des listes très longues.

10. Fonction chargement_glouton_2

```
</> Code Python
1 def chargement_glouton2(liste: list, capacite: int) -> list:
2     """
3     Renvoie :
4         liste des colis à charger pour maximiser le poids total
5         sans dépasser la capacité
6     """
7     colis_a_charger = []
8     poids_actuel = 0
9
10    for colis in liste:
11        if poids_actuel + colis.poids <= capacite:
12            colis_a_charger.append(colis)
13            poids_actuel += colis.poids
14
15    return colis_a_charger
```

Exercice 3 ►

/8 points

1. Le type approprié pour l'attribut **annee** est INT (entier) car c'est une année de naissance qui est représentée par un nombre entier (ex: 2012, 2013, etc.).
2. Une contrainte de domaine pertinente serait de limiter les valeurs d'année à un intervalle réaliste, par exemple:
 - **annee** >= 2007. (année de naissance raisonnable pour des enfants de 0 à 18 ans)
 - **annee** <= 2025. (année actuelle)

On pourrait donc écrire:

```
CHECK (annee >= 2007 AND annee <= 2025)
```

3. L'attribut **num_parent** suit une contrainte de référence. Il fait référence à l'attribut **tel** de la table **parent** (clé étrangère).
4. La clé primaire proposée est **tel** (numéro de téléphone).

Justification:

- Chaque parent a un numéro de téléphone unique
 - Il est utilisé comme référence dans la table enfant (**num_parent**)
 - Il identifie de manière unique chaque parent
 - Alternative possible: créer un attribut **id_parent** auto-incrémenté, mais **tel** semble être l'identifiant naturel choisi dans ce schéma
5. Cette requête lève une erreur car:
 - La table enfant a une contrainte de clé étrangère sur l'attribut **num_parent**
 - La valeur 33600782812 est référencée dans la table **enfant**
 - La modification de cette valeur dans la table **parent** entraînerait des références orphelines dans la table **enfant**
 - La base de données refuse cette modification pour maintenir l'intégrité référentielle

6.

```
INSERT INTO parent VALUES ('Bauges', 33600782812, 73340);
UPDATE enfant SET num_parent = 33619782812 WHERE num_parent = 33600782812;
DELETE FROM parent WHERE tel = 33619782812
```

7. Enfants nés avant 2014, triés par année : 2009, 2012, 2012, 2013

```
Nakamura
Hawa
Kian
Adrien
```

8.

```
SELECT enfant.prenom
FROM enfant
WHERE enfant.num_parent = 3619861122
ORDER BY enfant.prenom;
```

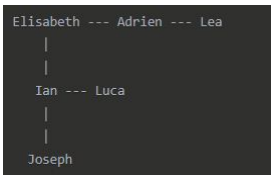
9.

```
SELECT enfant.id, enfant.prenom
FROM enfant
JOIN parent ON enfant.num_parent = parent.tel
WHERE parent.codep = 38520;
```

10. Un graphe non orienté suffit car:

- Les relations de "mésentente" entre enfants sont symétriques
- Si l'enfant A ne s'entend pas avec l'enfant B, alors B ne s'entend pas avec A
- Il n'y a pas de notion de direction dans cette relation
- On cherche simplement à éviter que deux enfants qui ne s'entendent pas soient dans le même groupe

11.

12. Fonction `degre`

</> Code Python

```
1 def degre(g: dict, s: str) -> int:
2     """
3     Renvoie le degré du sommet dans le graphe
4     """
5     return len(g[s])
```

13. Fonction `sommets_tries`

</> Code Python

```
1 def sommets_tries(g: dict) -> list:
2     sommets = [sommet for sommet in g]
3     n = len(sommets)
4     for i in range(1, n):
5         sommet_courant = sommets[i]
6         j = i-1
7         while j >= 0 and degre(g, sommets[j]) < degre(g, sommet_courant):
8             sommets[j+1] = sommets[j]
9             j = j - 1
10        sommets[j+1] = sommet_courant
11    return sommets
```

14. Tri utilisé : Tri par insertion

Coût d'exécution:

- Coût de la fonction `sommets_tries`: $O(n^2)$ - quadratique dans le pire des cas
- La fonction `degre` est de coût constant (hypothèse donnée)
- Le tri par insertion compare chaque élément avec tous les éléments précédents dans le pire cas

15. Analyse du graphe `g1` pour le coloriage:

- Octavie (degré 3) : couleur 0
- Pierre (degré 2) : couleur 1
- Raphael (degré 2) : couleur 2
- Virgile (degré 3) : couleur 1
- Elise (degré 2) : couleur 2
- Sixtine (degré 0) : couleur 0

16. Procédure `colorer_graphe`

Code Python

```
1 def colorer_graphe(g: dict, dc: dict):
2     # Pré-condition : les clés de dc sont les sommets
3     # de g, et les valeurs de dc sont toutes à -1
4     for s in dc:
5         couleur = plus_petite_couleur_hors_voisins(g, dc, s)
6         dc[s] = couleur
```

17. Code de la fonction `welsh_powell`

Code Python

```
1 def welsh_powell(g: dict) -> dict:
2     # initialisation à -1 pour tous les sommets dans le dictionnaire dc
3     dc = {sommet: -1 for sommet in g}
4     # coloration en suivant l'approche de Welsh-Powell
5     for sommet in sommets_tries(g):
6         couleur = plus_petite_couleur_hors_voisins(g, dc, sommet)
7         dc[sommet] = couleur
8     return dc
```